

EXPERIMENT-20:

Implement Future Sales Prediction using a suitable machine learning algorithm.

Code:

```
# =====  
# FUTURE SALES PREDICTION  
# Random Forest Regression  
# =====  
  
# 1. Import Libraries  
import pandas as pd  
import numpy as np  
from google.colab import files  
from sklearn.model_selection import train_test_split  
from sklearn.ensemble import RandomForestRegressor  
from sklearn.metrics import mean_absolute_error  
from sklearn.metrics import mean_squared_error  
from sklearn.metrics import r2_score  
print("Upload Sample - Superstore.xls")  
uploaded = files.upload()  
filename = list(uploaded.keys())[0]  
print("\nUploaded File:", filename)  
if filename.lower().endswith(".xls"):  
    !pip install -q xlrd  
    df = pd.read_excel(  
        filename,  
        engine="xlrd"  
    )  
elif filename.lower().endswith(".xlsx"):  
    !pip install -q openpyxl  
    df = pd.read_excel(  
        filename,
```

```

        engine="openpyxl"
    )
elif filename.lower().endswith(".csv"):
    df = pd.read_csv(filename)
else:
    raise ValueError(
        "Please upload .xls, .xlsx or .csv file"
    )
print("\nFirst 5 Rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
print(df.columns.tolist)
df.columns = (
    df.columns
    .str.strip()
)
if "Order Date" in df.columns:

    df["Order Date"] = pd.to_datetime(
        df["Order Date"],
        errors="coerce"
    )

    df["Order_Year"] = (
        df["Order Date"].dt.year
    )

    df["Order_Month"] = (

```

```

        df["Order Date"].dt.month
    )
    df["Order_Day"] = (
        df["Order Date"].dt.day
    )
    df["Order_DayOfWeek"] = (
        df["Order Date"].dt.dayofweek
    )
    # Remove original date
    df.drop(
        "Order Date",
        axis=1,
        inplace=True
    )
if "Ship Date" in df.columns:
    df["Ship Date"] = pd.to_datetime(
        df["Ship Date"],
        errors="coerce"
    )
    df["Ship_Year"] = (
        df["Ship Date"].dt.year
    )
    df["Ship_Month"] = (
        df["Ship Date"].dt.month
    )
    df["Ship_Day"] = (
        df["Ship Date"].dt.day
    )
    # Remove original date
    df.drop(

```

```

        "Ship Date",
        axis=1,
        inplace=True
    )
if "Sales" not in df.columns:

    raise ValueError(
        "Sales column not found."
    )
remove_columns = [

    "Row ID",
    "Order ID",
    "Customer ID",
    "Customer Name",
    "Product ID",
    "Product Name",
    "Postal Code",
    "Country"

]
for col in remove_columns:
    if col in df.columns:
        df.drop(
            col,
            axis=1,
            inplace=True
        )

```

```

numeric_columns = [

    "Sales",

    "Quantity",

    "Discount",

    "Profit"

]

for col in numeric_columns:

    if col in df.columns:

        df[col] = pd.to_numeric(

            df[col],

            errors="coerce"

        )

for col in df.columns:

    if df[col].dtype == "object":

        mode_value = df[col].mode()

        if len(mode_value) > 0:

            df[col] = df[col].fillna(

                mode_value[0]

            )

        else:

            df[col] = df[col].fillna(

                "Unknown"

            )

    else:

        df[col] = df[col].fillna(

            df[col].median()

        )

categorical_columns = df.select_dtypes(

```

```

        include=["object"]
    ).columns

df = pd.get_dummies(
    df,
    columns=categorical_columns,
    drop_first=True
)
datetime_columns = df.select_dtypes(
    include=["datetime"]
).columns

if len(datetime_columns) > 0:

    df.drop(
        columns=datetime_columns,
        inplace=True
    )
X = df.drop(
    "Sales",
    axis=1
)
y = df["Sales"]
X = X.astype(float)
y = y.astype(float)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,

    test_size=0.20,

```

```

    random_state=42
)
print("\n=====")
print("DATA SPLIT")
print("=====")
print(
    "Training Data:",
    X_train.shape
)
print(
    "Testing Data:",
    X_test.shape
)
model = RandomForestRegressor(
    n_estimators=100,
    random_state=42,

    n_jobs=-1
)
print("\nTraining model...")
model.fit(
    X_train,
    y_train
)
print("Training completed.")
y_pred = model.predict(
    X_test
)
mae = mean_absolute_error(

```

```

    y_test,
    y_pred
)

mse = mean_squared_error(
    y_test,
    y_pred
)
rmse = np.sqrt(mse)

r2 = r2_score(
    y_test,
    y_pred
)
print("\n=====")
print("FUTURE SALES PREDICTION RESULTS")
print("=====")

print(
    "MAE :",
    round(mae, 2)
)
print(
    "MSE :",
    round(mse, 2)
)
print(
    "RMSE :",
    round(rmse, 2)
)

```



```

***
      Product Name      Sales  Quantity \
0      Bush Somerset Collection Bookcase 261.9600      2
1 Hon Deluxe Fabric Upholstered Stacking Chairs,... 731.9400      3
2 Self-Adhesive Address Labels for Typewriters b... 14.6200      2
3      Bretford CR4500 Series Slim Rectangular Table 957.5775      5
4      Eldon Fold 'N Roll Cart System 22.3680      2

```

```

      Discount      Profit
0      0.00      41.9136
1      0.00      219.5820
2      0.00      6.8714
3      0.45     -383.0310
4      0.20      2.5164

```

[5 rows x 21 columns]

Dataset Shape:
(9994, 21)

Column Names:
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',

```

=====
DATA SPLIT
=====
Training Data: (7995, 614)
Testing Data: (1999, 614)

```

Training model...
Training completed.

```

=====
FUTURE SALES PREDICTION RESULTS
=====
MAE   : 87.06
MSE   : 247377.04
RMSE  : 497.37
R2 Score : 0.5812

```

```

=====
SALES PREDICTION
=====
Predicted Sales: 185.3

```